OS ORANGE PROBLEM

NAME: GAGAN R
SRN: PES1UG24CS665
SECTION: K

Screenshot-1(A)

```
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot-1(B)

```
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

Screenshot-2(A)

```
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot-2(B)

```
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d421
13ea09227ed0221bbe9db5e5c4b9aafc12|head -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$
```

Screenshot-3(A)

```
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$ echo "hello" > file1.txt
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$ echo "world" > file2.txt
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$ ./pes add file1.txt file2.txt
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  tree.c
  untracked:  test_objects.c
  untracked:  object.c
  untracked:  commit.c
  untracked:  Makefile
  untracked:  test_sequence.sh
  untracked:  pes.h
  untracked:  commit.h
  untracked:  README.md
  untracked:  index.h
  untracked:  pes.c
  untracked:  test_tree.c
  untracked:  index.c
  untracked:  tree.h
```

Screenshot-3(B)

```
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776322881 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776322891 6 file2.txt
```

Screenshot-4(A)

```
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$ make clean
make

./pes init
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$ echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"
Committed: 85101dca1b19... Initial commit
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$ echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"
Committed: c4cc3a6c3840... Add world
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$ echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"
Committed: dbbecc8861d4... Add farewell
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$ ./pes log
commit dbbecc8861d41d1e3f148995d9e6691876c27607c76a8a3909a002903963f745
Author: Gagan <PES1UG24CS665>
Date:   1776325441

    Add farewell

commit c4cc3a6c38408b251bdc785ca208b4396d4ab640e5fdffb877e43243f2a37f40
Author: Gagan <PES1UG24CS665>
Date:   1776325437

    Add world

commit 85101dca1b1988c3b28066de1ff98652ef57d01f31c50843539b54a06d4a9135
Author: Gagan <PES1UG24CS665>
Date:   1776325431

    Initial commita

gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$
Not Comm
```

Screenshot-4(B)

```
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/85/101dca1b1988c3b28066de1ff98652ef57d01f31c50843539b54a06d4a9135
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/c4/cc3a6c38408b251bdc785ca208b4396d4ab640e5fdffb877e43243f2a37f40
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/db/becc8861d41d1e3f148995d9e6691876c27607c76a8a3909a002903963f745
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$
```

Screenshot-4(C)

```
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$ cat .pes/refs/heads/main
dbbecc8861d41d1e3f148995d9e6691876c27607c76a8a3909a002903963f745
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
gagan-raghavan@gagan-raghavan-ThinkPad-E16-Gen-1:~/Documents/PES1UG24CS665-pes-vcs$
```

Q5.1: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement `pes checkout <branch>` — what files need to change in `.pes/`, and what must happen to the working directory? What makes this operation complex?

Ans:

Checkout means switching to another branch by updating references and restoring files.

- Update `.pes/HEAD` → point to new branch
- Read commit hash from `.pes/refs/heads/<branch>`
- Recreate working directory from that commit's tree

Working directory must:

- Remove old files
- Create/update new files

Complex because:

- Must handle file changes safely
- Should not overwrite uncommitted changes

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.
Ans:

When switching branches, we must ensure no important changes are lost. This is done by comparing the working directory with the index.

- For each file in the index:
  - Compare file size
  - Compare modification time (`mtime`)

If any difference is found:

- The file is considered modified (dirty)

If such a file would be overwritten during checkout:

- The operation must be aborted

This ensures user changes are not accidentally lost.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Ans:
A detached HEAD means that `HEAD` points directly to a commit hash instead of a branch.

- New commits can still be created
- But no branch points to them

So:

- These commits become unreachable if the user switches away

To recover:

- Create a new branch pointing to that commit

- This makes the commits reachable again

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Ans:

Over time, many objects (blobs, trees, commits) may become unused. Garbage collection removes these.

The process works as follows:

- Start from all branch heads (`refs/heads/*`)
- Traverse all reachable commits, trees, and blobs
- Mark all visited objects as reachable
- Delete all unmarked objects

To track reachable objects efficiently:

- Use a hash set for constant-time lookup

For a repository with:

- 100,000 commits and 50 branches

We may need to visit:

- Around 100k–300k objects (including trees and blobs)

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Ans:

Running garbage collection during a commit can cause a race condition.

- GC identifies an object as unused
- At the same time, a commit is being created that will reference that object
- GC deletes it before commit finishes

Result:

- The commit becomes corrupt or incomplete

To avoid this, Git:

- Uses locks to prevent concurrent conflicts
- Writes objects safely before updating references
- Delays deletion of objects until it is safe